
ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTY OF ELECTRICAL AND COMPUTER ENGINEERING
SOFTWARE ENGINEERING II
CODING STANDARDS — 2026

What this workshop is about

In this workshop, students will work in **groups of up to 2 members** (individual submissions are also allowed). Each team will fork a real, intentionally flawed repository, select a static analysis tool (a linter), and refactor the codebase until the linter reports zero violations on the modified files.

Requirements

- Visual Studio Code
- Python [Download Python](#)
- Java 8 or newer JRE/JDK
- Eclipse IDE for Java Developers [Download from Eclipse](#) | [Get executable](#)
- Git

Objectives:

- Configure and run an automated code quality tool against a real codebase.
- Produce an HTML report listing violations classified by severity.
- Apply naming, structure, and exception-handling conventions until violations on the changed files reach zero.

Introduction

Code conventions and best practices are guidelines that help **maintain consistency** and readability in **codebases**. Adhering to these standards improves code quality, makes collaboration easier, and reduces the likelihood of errors. While humans can manually enforce these conventions, it can be a tedious task. That's where **automated tools** come in.

Automated tools, such as **linting tools**, analyze source code to identify deviations from coding standards and best practices. They can check various aspects of code, including class design, method design, code layout, and formatting. Additionally, they can generate project-wide reports summarizing the issues found.

These tools provide a wide range of checks that developers can apply to their codebase:

- Annotations
- Block Checks
- Class Design
- Coding
- Headers
- Imports
- Javadoc Comments
- Metrics

- Miscellaneous
- Modifiers
- Naming Conventions
- Regexp
- Size Violations
- Whitespace

In this lab, we will focus on improving code quality and readability using a code standard tool.

Activity (step by step)

Step 1 — Pick a language and fork the matching repository

1. Begin by selecting the programming language you'll be working with for the workshop. You can choose either Python or Java based on your familiarity or interest
2. Read carefully the requirements in each repo.

Language	Fork this	Working file
Python	https://github.com/NataliaMw/loan-eligibility-python	src/loan/eligibility.py
Java	https://github.com/NataliaMw/loan-eligibility-java	src/main/java/ec/cooperativa/loan/Eligibility.java

The fork must be public. The fork name follows the convention is2-coding-standards-`<group>-<lang>`.

Step 2 — Pick a linter and document it in the README

Pick, or research by your own, one tool from the list below and declare it in the README, with version number and rule profile.

Language	Allowed tools
Python	Pylint, Flake8, SonarLint
Java	Checkstyle, PMD, SonarLint

The tool must produce an HTML (or web) report that classifies violations by severity.



Challenge (+5 points): configure a GitHub Actions workflow that runs the linter on every pull request to main and fails the build when violation count on changed files is greater than zero.

Step 3 — Run the linter on the unmodified fork and save the initial report

Before making any modifications to the codebase, you must run the linter against your forked repository in its current state. This establishes a baseline report of the existing code quality.

- **Execute the Linter:** Run the provided linting tool against the fork as-is (do not apply any fixes yet).
- **Save the Report:** Export and save the full HTML report to the following path inside your repository:
`reports/initial.html`.
- **Document Findings:** Take a screenshot of the violation overview section from this initial report and embed it directly into your PDF brief.

Step 4 — Fix violations in small commits

Systematically resolve the identified linting violations. Approach this iteratively to ensure your version control history remains clean and traceable.

- **Commit Incremental Fixes:** Address violations using small, focused commits. Do not bundle unrelated fixes into a single commit.
- **Format Commit Messages:** Every commit message must explicitly name the specific rule identifier addressed. For example:
 - `pylint(R0902): split converter into parse and format helpers`
 - `checkstyle(MagicNumber): extract validator coefficient constants`
 - `pmd(MethodLength): extract digit-sum loop into helper.`

Step 5 — Keep the functional contract of the module

Your corrected module must continue to satisfy the exact business logic that the fork currently implements.

- **No Scope Creep:** Do not introduce new features, and **do not delete existing tests** to make the linter happy.

- **Business Logic:** The module evaluates whether a member of a *cooperativa de ahorro y crédito* is eligible for a loan. It must return the eligibility decision, the maximum loan amount, the interest rate, and a list of reason codes explaining any rejections.
- **Verification:** Existing tests document this contract. Running the test suite before and after your refactoring is your proof that the behavior has been perfectly preserved.

Step 6 — Run the linter again and save the final report

Run the linter one last time on your corrected code.

- **Save the Report:** Export the final results to `reports/final.html`. The expected outcome is **zero violations** on your changed files.
- **Document Final Success:** Embed a screenshot of this clean overview into your PDF brief.

Note: Keep in mind that automated tools won't catch every standard code issue. It's your responsibility to maintain clean code and follow best practices, including appropriate naming conventions.

Deliverables

Only **one submission per group** is required on Aula Virtual. Ensure all group members' names are on the PDF brief.

Item	Format	Where
PDF brief with introduction, development, conclusions, recommendations	PDF	Aula Virtual
Initial linter report	HTML in reports/initial.html	Git repository
Final linter report	HTML in reports/final.html	Git repository
Screenshots of the process	PNG in docs/figures/	Git repository
Public repository URL	URL inside the PDF brief	Aula Virtual

Rubric (60 + 40)

Compliance — 60 points

Item	Points
Repository is public and accessible	10
Initial report present and matches the embedded screenshot	10

Item	Points
Final report shows zero violations on changed files	25
Existing test suite still green after the refactor	10
Suppressions documented with rule id and reason	5

Mastery — 40 points

Item	Points
PDF brief content: introduction, development, conclusions, recommendations	15
Naming, structure, and exception handling demonstrate the rule catalog vocabulary in commit messages	15
Screenshots present, legible, and referenced from the brief	5
Optional GitHub Actions workflow that enforces the violation count	5

Late Submission Policy

Delay (§)	Penalty (Ω)
1 hour or less	loss of 10%
1 to 6 hours	loss of 20%
6 to 24 hours	loss of 30%
Over 24 hours:	loss of 100%

(§) every clock hour counts including weekends or holidays.

(Ω) automatic and non-negotiable penalty